

Cadence — the whole-project map

Every part of the project sorted into labelled drawers, all hung off one data-flow spine. Colour key: **teal**=walk · **amber**=freeze/cue · **green**=still/ok · grey=legacy/shelved.



Everything below is just a *part* of this spine — the drawers tell you what each part is and where it lives. The cue is *debounced* (2 windows on / 2 off) so a one-off bump never buzzes — that latched stream is what the garment actually emits.

1 HARDWARE — the physical stack

- **Circuit Playground Express (CPX)** — worn on the **ankle** (distal lower limb, where the FoG tremor is largest). SAMD21 chip, **LIS3DH** 3-axis accelerometer (the only sensor that matters here), **10-pixel NeoPixel ring** (the lights), **button A** (press to calibrate), on-board **vibration motor — the cue actuator**: it buzzes the rhythmic stepping prompt when a freeze is detected. That buzz *is* the closed loop.
- **ESP32** — the radio bridge. CPX --UART/Serial1--> ESP32 --Wi-Fi/TCP--> laptop. The band is untethered.
- **Laptop / Pi** — the TCP server (fixed address 0.0.0.0:8765); runs the live dashboard and the offline analysis. The ESP32 dials in as a client.
- **Legacy**: direct USB tether (no ESP32) is kept only for bench bring-up.

2 TWO DEVICE MODES — the project's lineage

Wrist — explored, then shelved

Open-loop clinical tremor monitor: watch for the 4–6 Hz resting tremor and *report* it (measure-only, no buzzing back). A reasonable idea, but *not* what we built — kept here so the lineage stays honest.

Ankle — what you build & demo

Closed-loop freeze-of-gait garment: detect a freeze → fire a rhythmic vibrotactile cue to help the wearer step off again. This is where the CNN + Daphnet data live — the device the leaflet, worksheet and live demo are about.

3 LABEL SCHEMES — the three sets of “categories” (don't mix them up)

Where it lives	Called	How many	The actual categories
Training data Daphnet, ankle	annotation codes	3	0 =not in experiment (<i>dropped</i>) · 1 =no-freeze · 2 =freeze
The ML model 1-D CNN, offline	classes	2	no_freeze · freeze
The live band on the ankle, demo	states	3	WALK · FREEZE · STILL
The dataset's 3 codes collapse into the model's 2 classes (code 0 thrown away; a 4 s window is freeze if >50 % of its samples are code 2). On the live band, FREEZE is the model's call; a movement-energy gate then splits the non-freeze time into WALK (moving) and STILL (quiet limb).			

4 TWO ML APPROACHES — how a window becomes a label

Option A — features → classic ML

Boil each window down to **9 hand-picked numbers** (band powers, Freeze Index, tremor power, RMS, jerk ...) then feed a **Random Forest**.

Role: the *interpretable baseline* — lets you say *which* feature mattered (SHAP). Not deployed.

Option B — raw window → CNN (yours)

Feed the **raw** 3-axis window (3×256) straight into a small **1-D CNN** (FoGNet, ~18–30k params); it learns its own filters.

Role: *your trained model* — the deployed detector. Its leave-one-subject-out sens / spec on Daphnet are the headline ML result.

5 DATA

- **Daphnet FoG** — UCI, *ankle*, 64 Hz. The **primary** dataset; trains & validates the CNN. *Authorised to use*.
- **Your own ~40s ankle capture** (still/walk/freeze) — the honest source for the **worksheet's** headline Freeze-Index numbers.
- **PADS** — PhysioNet, *wrist* (Apple Watch, 100 Hz). Only relevant to the shelved wrist branch; *not downloaded*.

6 METRICS

- **Sensitivity** — of the real events, how many did we catch? (a miss is the costly error)
- **Specificity** — when nothing's happening, how often do we stay quiet? (no crying wolf)
- **Confusion matrix** — the 2x2 the two rates come from.
- *Not accuracy* — events are rare, so “always say no” scores ~90 % and means nothing.

7 SOFTWARE — the repo, by job

- `pi/fog/config.py` — single source of truth: sample rate, window sizes, bands, label map, protocol.
- `pi/fog/dsp.py` — the maths: filter, magnitude, band power, Freeze Index, tremor power (*torch-free*).
- `pi/fog/model.py` — the 1-D CNN (FoGNet); the *only* file that imports torch.
- `pi/fog/metrics.py` — sensitivity / specificity + the Freeze-Index baseline.
- `colab/fog_allinone.py` — trains the CNN + runs the full LOSO + debounce; `colab/pick_threshold.py` picks the deploy threshold honestly (nested LOSO); `pi/make_accuracy_figs.py` builds the Option-A baseline + figures.
- `firmware/variants/*.ino` — on-board code: `cpx_fog_*` (ankle, deployed), `cpx_tremor_monitor` (wrist, shelved), ESP32 bridge/hub.
- `dashboard.html` — the live screen: states + freeze/cue badge + CSV recorder.
- `worksheet/*.tex` — this map, the theory explainers, the demo sheet, the accuracy worksheet.

8 GRADED DELIVERABLES — due Fri (all three are marked)

- **Accuracy worksheet** — headline metric is **sensitivity & specificity**, *not* accuracy. Numbers must be **real**: from your own ankle capture (worksheet headline) and Daphnet LOSO (the CNN).
- **Leaflet / advert** — the ankle freeze-of-gait cueing garment, in plain language.
- **Project Reflection** — **Category 1: NO AI allowed**. Write it entirely yourself; any AI use is academic misconduct. (AI help on design/code is fine — this isn't.)

One-line filing rule: if someone says “the 3 categories” they mean the live **WALK/FREEZE/STILL** states (drawer 3, bottom row); “the 2 classes” is the CNN (drawer 3, middle row); the dataset's “3 codes” are raw labels that collapse into those 2. Frequency *bands* (loco / freeze / tremor) are Hz ranges used to build features — they are inputs, never output categories.